

LAB WORKBOOK

Assignment 8: Data-Driven Automation (DDT)

Tier 3 – Framework Design & Test Organization

1. Module Name and Experiment Title

Module: Unit Test Frameworks(Framework Design & Test Organization)

Experiment: Data-Driven Login Automation

2. Problem Statement

Build a Selenium login automation script that reads multiple test cases from an external source, such as a CSV file, and executes the same test logic for each data combination. The test data must include correct and incorrect usernames and passwords, and the automation must verify that the appropriate validation error is displayed for invalid credentials.

3. Objective

To implement data-driven testing by separating test data from test logic and executing multiple positive and negative login scenarios using PyTest parameterization.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Test Framework	PyTest
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Application Under Test	Automation Exercise (https://automationexercise.com/)
External Data Source	CSV file
Python Module	csv / csv.DictReader
Concepts	Data-driven testing, parameterization, positive/negative testing, reusable test data

5. Source Code / Implementation

5.1 Test Data – test_data/login_test_data.csv

```
username,password,expected
YOUR_VALID_EMAIL,YOUR_VALID_PASSWORD,success
invalid@example.com,wrongpassword,error
wrong@example.com,wrongpassword,error
,YOUR_VALID_PASSWORD,error
YOUR_VALID_EMAIL,,error
```

5.2 CSV Reader – utilities/csv_reader.py

```
import csv

def read_csv(file_path):
    test_data = []

    with open(file_path, mode="r", newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            test_data.append(row)

    return test_data
```

5.3 Data-Driven Test – tests/test_login_ddt.py

```
import os
import pytest

from pages.login_page import LoginPage
from utilities.csv_reader import read_csv

CSV_PATH = os.path.join(
    os.path.dirname(os.path.dirname(__file__)),
    "test_data",
    "login_test_data.csv"
)

test_data = read_csv(CSV_PATH)

@pytest.mark.parametrize("data", test_data)
def test_login_data_driven(driver, data):

    login_page = LoginPage(driver)

    username = data["username"]
    password = data["password"]
    expected = data["expected"]

    login_page.login(username, password)

    if expected == "success":
        assert "Logged in as" in driver.page_source
    else:
        error_message = login_page.get_login_error()
        assert "incorrect" in error_message.lower()
```

6. Implementation Steps

1. Create a CSV file containing valid and invalid username/password combinations.
2. Use `csv.DictReader()` to read each row as a dictionary.
3. Load the test data before test execution.
4. Use `@pytest.mark.parametrize()` to pass each CSV row to the same test function.
5. Enter the username and password through the `LoginPage` object.
6. For valid credentials, verify that the logged-in state is displayed.
7. For invalid credentials, verify that the login validation error is displayed.

8. Execute all data combinations through PyTest and review the individual results.

7. Result

Data-driven login automation was successfully implemented using an external CSV file and PyTest parameterization. Multiple credential combinations were executed using a single reusable test function.

8. Observation

Separating test data from test logic makes it possible to add or modify test cases without changing the core Selenium test implementation. PyTest generates a separate result for each parameterized data set.

9. Conclusion

This experiment demonstrated data-driven testing with Selenium, Python CSV handling, and PyTest parameterization. The approach improves test coverage and reduces duplication when many input combinations must be validated.

10. Execution Evidence – Illustrative Screenshots

The following visuals are illustrative documentation screenshots. Replace them with actual screenshots from your own execution before submitting if real execution evidence is required.

Figure 1: External CSV test data used for data-driven execution.

4	wrong@example.com	wrongpassword	error
5		wrongpassword	error
6	validuser@example.com		error

Fig 4: Test Data File (login_test_data.csv)

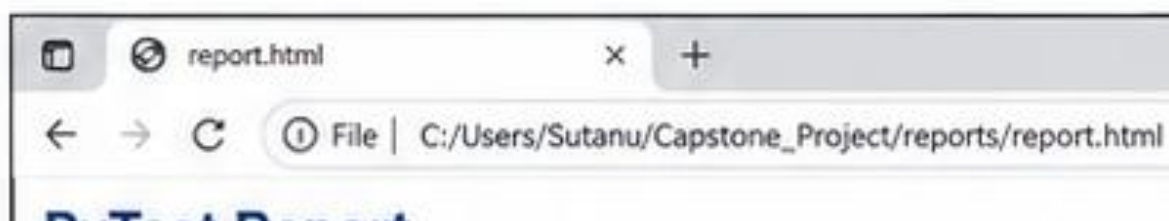


Figure 2: PyTest parameterized data-driven test.

```
11 test_data = read_csv(CSV_PATH)
12
13 @pytest.mark.parametrize('data', test_data)
14 def test_login_data_driven(driver, data):
15     login_page = LoginPage(driver)
16
17     username = data['username']
18     password = data['password']
19     expected = data['expected']
20
21     login_page.login(username, password)
22     if expected == 'success':
23         assert 'Logged in as' in driver.page_source
24     else:
25         error_message = login_page.get_login_error()
26         assert 'incorrect' in error_message.lower()
```

tests\test_log
tests\test_pro
tests\test_uni
tests\test_log
tests\test_log
tests\test_log
=====
Generated html
eport.html
C:\Users\Sutanu

Fig 5: test_login_ddt.py (Data Driven Test)



Figure 3: PyTest terminal execution showing parameterized cases.

```
login.py ..... PASSED [ 16%]
duct_search.py ..... PASSED [ 33%]
ttest_login.py ..... PASSED [ 50%]
in_ddt.py::test_login_data_driven[data0] . PASSED [ 66%]
in_ddt.py::test_login_data_driven[data1] . PASSED [ 83%]
in_ddt.py::test_login_data_driven[data2] . PASSED [100%]
===== 6 passed in 58.23s =====
report: file:///C:/Users/Sutanu/Capstone_Project/reports/r
hu\Capstone_Project>
```

Fig 6: PyTest Execution in Terminal

Capstone_Project > screenshots